

STRUTTURE DATI: LE LISTE

Definizione:

una lista è una struttura dati che rappresenta una collezione di elementi disposti in ordine lineare. Può essere utilizzata per memorizzare dati in modo dinamico, permettendo l'aggiunta, la rimozione e l'accesso agli elementi in modo flessibile. A differenza di un array (statico), la lista non richiede di definire una dimensione fissa, poiché la memoria viene allocata dinamicamente.

Qual è il significato di allocazione dinamica in riferimento ad una lista?

In una lista, l'allocazione dinamica della memoria significa che lo spazio per gli elementi (**NODI**) viene richiesto al momento dell'aggiunta di nuovi elementi, consentendo alla lista di crescere senza una dimensione predefinita. Questo è particolarmente utile, perché a differenza di un array, non è necessario stabilire in anticipo quanti elementi verranno memorizzati.

1. **Creazione di un nodo:** quando un elemento viene aggiunto alla lista, viene creato un nuovo nodo in memoria.

```
Nodo nuovoNodo = new Nodo(10);
```

```
/*
```

```
    Mediante il costruttore si alloca dinamicamente lo spazio
    per memorizzare il valore 10 e il collegamento al prossimo
    nodo.
```

```
*/
```

2. **Espansione dinamica:** la lista non ha limiti rigidi: puoi continuare ad aggiungere elementi fino a quando c'è memoria disponibile nel sistema. Ogni nuovo nodo viene allocato nel momento in cui serve.
3. **Collegamento tra nodi:** ogni nodo contiene un riferimento (puntatore) al nodo successivo. La memoria per il collegamento viene allocata dinamicamente durante l'esecuzione.

Cos'è un nodo?

Un nodo è l'unità di base di una lista. Ogni nodo contiene due parti:

- **Dati (info):** il valore o l'informazione memorizzata nel nodo.
- **Collegamento (link):** un riferimento al nodo successivo nella lista (o **null** se il nodo è l'ultimo).

Cos'è un collegamento?

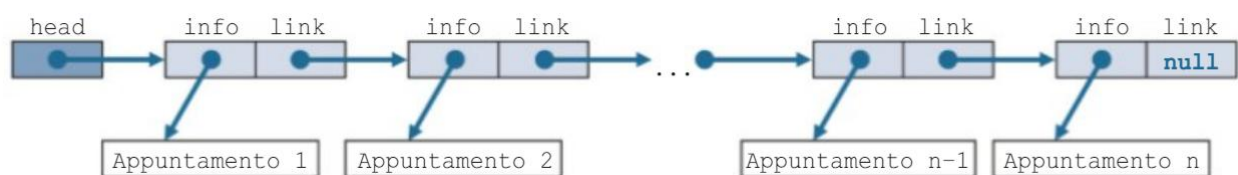
Il collegamento è il meccanismo che unisce un nodo al successivo, creando una catena. Grazie a questi collegamenti, è possibile attraversare la lista in ordine (in modo sequenziale).

Tipi di liste

Esistono vari tipi di liste, tra cui:

- **Lista semplicemente collegata:** ogni nodo ha un collegamento al successivo.
- **Lista doppiamente collegata:** ogni nodo ha un collegamento sia al nodo precedente che a quello successivo.
- **Lista circolare:** la lista può essere semplicemente o doppiamente collegata, ma l'ultimo nodo è collegato al primo, formando un ciclo.

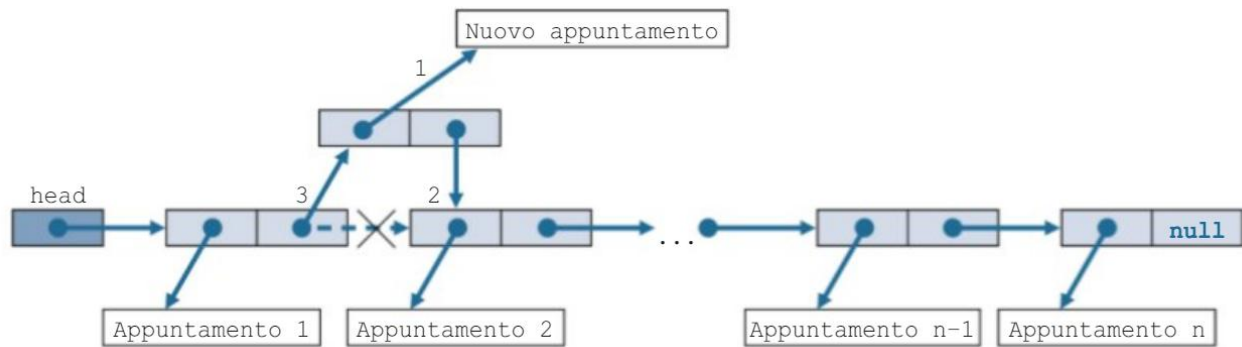
Rappresentazione di una lista



- **head** è il riferimento d'ingresso alla struttura.

Per inserire un nuovo appuntamento come secondo elemento della lista (per esempio per mantenere la lista ordinata cronologicamente) si opera come segue:

1. si crea un nuovo nodo la cui componente **info** riferisce il nodo Nuovo appuntamento.
2. Si imposta la componente **link** del nuovo nodo in modo che riferisca al nodo relativo ad appuntamento 2.
3. Si imposta la componente **link** del nodo relativo ad appuntamento 1 in modo che riferisca al nodo relativo a Nuovo appuntamento, eliminando il preesistente collegamento tra i nodi relativi ad Appuntamento 1 e Appuntamento 2.



ARRAY E LISTE A CONFRONTO

Caratteristica	Array	Lista
Definizione	Una struttura dati con una sequenza di elementi memorizzati in posizioni contigue di memoria.	Una struttura dati dinamica composta da nodi collegati tramite riferimenti.
Dimensione	Fissa (deve essere specificata al momento della creazione).	Dinamica (si può espandere o ridurre durante l'esecuzione).
Accesso agli elementi	Accesso diretto (tempo costante utilizzando un indice).	Accesso sequenziale, occorre attraversare i nodi (tempo lineare, ovvero in funzione del numero di nodi).
Prestazioni di memoria	Occupazione di memoria ottimale (utilizza solo lo spazio richiesto per i dati).	Maggiore overhead di memoria a causa dei riferimenti aggiuntivi per i collegamenti.
Inserimento/Rimozione	Costoso poiché richiede lo spostamento degli elementi.	Efficiente se si conosce il nodo precedente (o si lavora in una lista doppiamente collegata).
Ordinamento	Più efficiente (le celle contigue favoriscono algoritmi di sorting come quicksort e mergesort).	Meno efficiente, poiché l'accesso ai nodi è sequenziale.
Uso tipico	Quando si conosce la dimensione iniziale dei dati e l'accesso diretto è prioritario.	Quando la dimensione dei dati varia frequentemente e le operazioni di inserimento/rimozione sono frequenti.

Def.(OVERHEAD): quantità di memoria extra, oltre alla memoria strettamente utilizzata per i dati stessi, necessaria per gestire e supportare una struttura dati o un processo.

Esempio pratico

Immagina di avere una lista collegata:

- ogni nodo memorizza un dato (ad esempio un numero intero).
- Oltre al dato, ogni nodo deve memorizzare anche un riferimento (o collegamento) al nodo successivo.

Quindi, la memoria totale utilizzata da un nodo in una lista collegata include:

1. **la memoria per il dato** (ad esempio un intero).
2. **la memoria per il riferimento** (ad esempio il puntatore al nodo successivo).

Questa memoria aggiuntiva per il riferimento è l'**overhead**.

L'overhead di memoria rappresenta lo spazio aggiuntivo necessario per gestire i riferimenti in una lista collegata. A differenza di un array, che conserva esclusivamente i dati senza alcun collegamento tra gli elementi, una lista collegata utilizza memoria extra per mantenere i collegamenti tra i nodi. Questo "costo" non è direttamente legato ai dati memorizzati, ma è dovuto ai meccanismi e alle strutture che garantiscono una gestione efficiente e dinamica della lista. È, in sostanza, un investimento di memoria che offre maggiore flessibilità e funzionalità rispetto a strutture dati più semplici.

Come sono memorizzati gli array di oggetti nella memoria?

- **L'array stesso (la struttura contigua):** sequenza contigua di riferimenti memorizzata in un'area specifica della memoria, chiamata **heap**. Ogni elemento dell'array è un puntatore (riferimento) a un oggetto. Questo significa che l'array, anch'esso un oggetto, non contiene direttamente i dati degli oggetti, ma solo i riferimenti ad essi.
- **Gli oggetti associati:** gli oggetti a cui puntano i riferimenti dell'array sono memorizzati separatamente, sempre nell'heap. Ogni oggetto occupa il proprio spazio in memoria e l'array si limita a fornire accesso a questi oggetti tramite i riferimenti.

ESEMPIO DI CODICE PER CREARE UNA CLASSE NODO E UNA CLASSE LISTA

```
public class Nodo {
    private int dato; // Valore memorizzato nel nodo
    private Nodo prossimo; // Collegamento al nodo successivo

    // Costruttore
    public Nodo(int dato) {
        this.dato = dato;
        this.prossimo = null; // Il prossimo è inizialmente null
    }

    // Getter e Setter
    public int getDato() {
        return dato;
    }

    public void setDato(int dato) {
        this.dato = dato;
    }

    public Nodo getProssimo() {
        return prossimo;
    }
}
```

```

    public void setProssimo(Nodo prossimo) {
        this.prossimo = prossimo;
    }
}

```

Struttura base della classe Lista.

```

public class Lista {
    private Nodo testa; // Riferimento al primo nodo della lista
    private int dimensione; // Numero totale di elementi nella lista

    // Costruttore
    public Lista() {
        testa = null; // La lista è inizialmente vuota
        dimensione = 0;
    }
    .....
    .....
}

```

VISITA DEGLI ELEMENTI DI UNA LISTA E STAMPA

```

public class Lista {
    private Nodo testa; //Primo nodo della lista

    public Lista() {
        // Creazione automatica di tre nodi iniziali
        Nodo nodo1 = new Nodo(10);
        Nodo nodo2 = new Nodo(20);
        Nodo nodo3 = new Nodo(30);

        // Collegamento dei nodi
        nodo1.setProssimo(nodo2);
        nodo2.setProssimo(nodo3);

        // Imposto il primo nodo
        testa = nodo1;
    }
}

```

```
// Metodo per visitare e stampare tutti i nodi della lista
public void stampaLista() {
    Nodo corrente = testa;
    while (corrente != null) {
        System.out.println("Nodo: " + corrente.getDato());
        corrente = corrente.getProssimo();
    }
}

public class Main {
    public static void main(String[] args) {
        // Creazione della lista con 3 nodi
        Lista lista = new Lista();

        // Stampiamo il contenuto della lista
        System.out.println("Contenuto della lista:");
        lista.stampaLista();
    }
}
```

INSERIMENTO DI UN ELEMENTO IN CODA

```
public class Nodo {
    private int dato;
    private Nodo prossimo;

    public Nodo(int dato) {
        this.dato = dato;
        this.prossimo = null;
    }

    public int getDato() {
        return dato;
    }

    public Nodo getProssimo() {
        return prossimo;
    }
}
```

```

    public void setProssimo(Nodo prossimo) {
        this.prossimo = prossimo;
    }
}

public class Lista {
    private Nodo testa; // Primo nodo della lista

    public Lista() {
        testa = null; // La lista parte vuota
    }

    // Metodo per aggiungere un nuovo nodo in coda
    public void aggiungiInCoda(int dato) {
        Nodo nuovoNodo = new Nodo(dato);
        if (testa == null) {
            // Se la lista è vuota, il nuovo nodo diventa la testa
            testa = nuovoNodo;
        } else {
            Nodo corrente = testa;
            while (corrente.getProssimo() != null) {
                corrente = corrente.getProssimo();
            }
            // Inserisce il nuovo nodo alla fine
            corrente.setProssimo(nuovoNodo);
        }
    }

    // Metodo per stampare la lista
    public void stampaLista() {
        Nodo corrente = testa;
        while (corrente != null) {
            System.out.print(corrente.getDato() + " -> ");
            corrente = corrente.getProssimo();
        }
        System.out.println("null"); // Indica la fine della lista
    }
}

```



```

public class Main {
    public static void main(String[] args) {
        Lista lista = new Lista();

        // Aggiungiamo alcuni elementi alla lista
        lista.aggiungiInCoda(5);
        lista.aggiungiInCoda(10);
        lista.aggiungiInCoda(15);

        // Stampiamo la lista dopo gli inserimenti
        System.out.println("Lista dopo l'inserimento:");
        lista.stampaLista();

        // Aggiungiamo un nuovo elemento in coda
        lista.aggiungiInCoda(20);

        // Stampiamo di nuovo la lista per vedere il risultato
        System.out.println("Lista con inserimento in coda: ");
        lista.stampaLista();
    }
}

```

INSERIMENTO DI UN ELEMENTO IN TESTA

```

public class Nodo {
    private int dato;
    private Nodo prossimo;
    public Nodo(int dato) {
        this.dato = dato;
        this.prossimo = null;
    }
    public int getDato() {
        return dato;
    }
}

```

```
public Nodo getNext() {
    return prossimo;
}

public void setNext(Nodo prossimo) {
    this.prossimo = prossimo;
}
}

public class Lista {
    private Nodo testa; // Primo nodo della lista

    public Lista() {
        // Creazione di tre nodi iniziali
        Nodo nodo1 = new Nodo(10);
        Nodo nodo2 = new Nodo(20);
        Nodo nodo3 = new Nodo(30);
        // Collegamento dei nodi
        nodo1.setNext(nodo2);
        nodo2.setNext(nodo3);
        // Imposto il primo nodo come testa della lista
        testa = nodo1;
    }

    // Metodo per inserire un nodo in testa alla lista
    public void inserisciInTesta(int dato) {
        // Creazione del nuovo nodo
        Nodo nuovoNodo = new Nodo(dato);
        // Collegamento del nuovo nodo alla lista
        nuovoNodo.setNext(testa);
        testa = nuovoNodo; // Il nuovo nodo diventa la testa
    }
}
```

```

// Metodo per stampare la lista
public void stampaLista() {
    Nodo corrente = testa;
    while (corrente != null) {
        System.out.print(corrente.getDato() + " -> ");
        corrente = corrente.getProssimo();
    }
    System.out.println("null"); //la fine della lista
}

}

public class Main {
    public static void main(String[] args) {
        // Creazione della lista con 3 nodi iniziali
        Lista lista = new Lista();
        // Stampo la lista originale
        System.out.println("Lista originale:");
        lista.stampaLista();
        // Inseriamo un nuovo nodo in testa
        lista.inserisciInTesta(5);
        // Stampo la lista dopo l'inserimento
        System.out.println("Lista dopo inserimento in testa: ");
        lista.stampaLista();
    }
}

```

ELIMINAZIONE DI UN ELEMENTO IN CODA

Lista iniziale:

- Contiene tre nodi con valori 10, 20 e 30.

Eliminazione in coda:

- Troviamo il penultimo nodo (quello prima dell'ultimo) e facciamo in modo che il suo riferimento (prossimo) punti a null.
- L'ultimo nodo (30) viene rimosso.

```
public class Nodo {  
    private int dato;  
    private Nodo prossimo;  
    public Nodo(int dato) {  
        this.dato = dato;  
        this.prossimo = null;  
    }  
    public int getDato() {  
        return dato;  
    }  
    public Nodo getProssimo() {  
        return prossimo;  
    }  
    public void setProssimo(Nodo prossimo) {  
        this.prossimo = prossimo;  
    }  
}
```

```
public class Lista {  
    private Nodo testa; // Primo nodo della lista  
    public Lista() {  
        // Creazione di tre nodi iniziali  
        Nodo nodo1 = new Nodo(10);  
        Nodo nodo2 = new Nodo(20);  
        Nodo nodo3 = new Nodo(30);  
        // Collegamento dei nodi  
        nodo1.setProssimo(nodo2);  
        nodo2.setProssimo(nodo3);  
        // Imposto il primo nodo come testa della lista  
        testa = nodo1;  
    }  
    // Metodo per eliminare l'ultimo nodo (in coda)  
    public void eliminaInCoda() {  
        if (testa == null) {  
            System.out.println("La lista è vuota!");  
            return; // Nessun nodo da eliminare  
        }  
        if (testa.getProssimo() == null) {  
            // Se c'è solo un nodo nella lista  
            testa = null; // La lista diventa vuota  
            return;  
        }  
        // Altrimenti, attraversiamo la lista per trovare il penultimo nodo  
        Nodo corrente = testa;  
        while (corrente.getProssimo().getProssimo() != null) {  
            corrente = corrente.getProssimo();  
        }  
    }  
}
```

```

        // Il penultimo nodo deve puntare a null (rimuoviamo l'ultimo)
        corrente.setProssimo(null);
    }
    // Metodo per stampare tutti gli elementi della lista
    public void stampaLista() {
        Nodo corrente = testa;
        while (corrente != null) {
            System.out.print(corrente.getDato() + " -> ");
            corrente = corrente.getProssimo();
        }
        System.out.println("null"); // la fine della lista
    }
}

public class Main {
    public static void main(String[] args) {
        Lista lista = new Lista(); // Creazione della lista

        // Stampo la lista originale
        System.out.println("Lista originale:");
        lista.stampaLista();
        // Eliminiamo l'ultimo nodo
        lista.eliminaInCoda();
        // Stampo la lista dopo l'eliminazione in coda
        System.out.println("Lista eliminazione in coda: ");
        lista.stampaLista();
    }
}

```

Spiegazione:

```
while (corrente.getProssimo().getProssimo() != null) {  
    corrente = corrente.getProssimo();  
}
```

1. `corrente.getProssimo()`:

- questo restituisce il riferimento al nodo successivo rispetto al nodo attualmente puntato da `corrente`, cioè punta al "nodo prossimo" nella lista.

2. `corrente.getProssimo().getProssimo()`:

- dal nodo successivo (`corrente.getProssimo()`), si accede al nodo che viene dopo, cioè il "nodo prossimo del prossimo".
- Serve per verificare se il prossimo nodo è **l'ultimo nodo della lista** (quello il cui `getProssimo()` è pari a `null`).

Obiettivo del ciclo

Il ciclo `while` ha come scopo attraversare la lista fino a fermarsi **al penultimo nodo**. Si usa quindi questa logica:

- Se **il nodo successivo al prossimo** (`getProssimo().getProssimo()`) è `null`, significa che il nodo successivo è l'ultimo nodo della lista.
- Il ciclo si interrompe in modo che `corrente` punti al penultimo nodo.

ELIMINAZIONE DI UN ELEMENTO IN TESTA

```
public class Nodo {
    private int dato;
    private Nodo prossimo;
    public Nodo(int dato) {
        this.dato = dato;
        this.prossimo = null;
    }
    public int getDato() {
        return dato;
    }
    public Nodo getProssimo() {
        return prossimo;
    }
    public void setProssimo(Nodo prossimo) {
        this.prossimo = prossimo;
    }
}

public class Lista {
    private Nodo testa; // Riferimento al primo nodo della lista

    public Lista() {
        // Creazione di tre nodi iniziali
        Nodo nodo1 = new Nodo(10);
        Nodo nodo2 = new Nodo(20);
        Nodo nodo3 = new Nodo(30);
        // Collegamento dei nodi
        nodo1.setProssimo(nodo2);
        nodo2.setProssimo(nodo3);
    }
}
```



```
// Imposta il primo nodo come testa della lista
testa = nodo1;
}
// Metodo per eliminare il nodo in testa
public void eliminaInTesta() {
    if (testa == null) {
        System.out.println("La lista è vuota!");
        return; // Nessun nodo da eliminare
    }
    // La testa diventa il nodo successivo
    testa = testa.getProssimo();
}
// Metodo per stampare il contenuto della lista
public void stampaLista() {
    Nodo corrente = testa;
    while (corrente != null) {
        System.out.print(corrente.getDato() + " -> ");
        corrente = corrente.getProssimo();
    }
    System.out.println("null"); // fine della lista
}
}
```

```

public class Main {
    public static void main(String[] args) {
        Lista lista = new Lista(); // Creazione della lista
        // Stampiamo la lista originale
        System.out.println("Lista originale:");
        lista.stampaLista();
        // Eliminiamo il primo nodo (in testa)
        lista.eliminaInTesta();
        // Stampiamo la lista dopo l'eliminazione in testa
        System.out.println("Lista eliminazione in testa: ");
        lista.stampaLista();
    }
}

```

INSERIMENTO IN POSIZIONE INTERMEDIA

La lista parte con tre nodi predefiniti e consente di aggiungere un nuovo elemento nella posizione desiderata.

```

public class Nodo {
    private int dato;
    private Nodo prossimo;
    public Nodo(int dato) {
        this.dato = dato;
        this.prossimo = null;
    }
    public int getDato() {
        return dato;
    }
    public Nodo getProssimo() {
        return prossimo;
    }
}

```

```

    public void setProssimo(Nodo prossimo) {
        this.prossimo = prossimo;
    }
}

public class Lista {
    private Nodo testa; // Primo nodo della lista
    //Costruttore: inizializza la lista con tre nodi predefiniti
    public Lista() {
        Nodo nodo1 = new Nodo(10);
        Nodo nodo2 = new Nodo(20);
        Nodo nodo3 = new Nodo(30);
        // Collegamento dei nodi
        nodo1.setProssimo(nodo2);
        nodo2.setProssimo(nodo3);
        // Imposta il primo nodo come testa della lista
        testa = nodo1;
    }
    //Metodo per inserire un nuovo elemento in una posizione specifica
    public void inserisciInPosizione(int posizione, int dato) {
        if (posizione <= 0) {
            System.out.println("Posizione non valida.");
            return; //Esco dal metodo
        }
        Nodo nuovoNodo = new Nodo(dato); //Creo un nuovo nodo
        if (posizione == 1) {
            // Inserimento in testa
            nuovoNodo.setProssimo(testa);
            testa = nuovoNodo;
            return; //Esco dal metodo
        }
    }
}

```

```

    /*
    Creo una variabile corrente di tipo Nodo, che inizialmente punta al
    primo nodo della lista (la testa)

    Questa variabile viene usata per attraversare la lista e trovare il
    punto esatto in cui inserire il nuovo nodo.
    */

    Nodo corrente = testa;

    int posizioneCorrente = 1;
    // Attraversiamo la lista fino al nodo prima della posizione desiderata
    while (corrente != null && posizioneCorrente < posizione - 1){
        corrente = corrente.getProssimo();
        posizioneCorrente++;
    }
    if (corrente == null) {
        System.out.println("La posizione supera la dimensione. ");
        return;
    }
    // Inserimento del nuovo nodo in posizione intermedia
    nuovoNodo.setProssimo(corrente.getProssimo());
    corrente.setProssimo(nuovoNodo);
}

// Metodo per stampare i nodi della lista
public void stampaLista() {
    Nodo corrente = testa;
    while (corrente != null) {
        System.out.print(corrente.getDato() + " -> ");
        corrente = corrente.getProssimo();
    }
    System.out.println("null"); // la fine della lista
}
}

```

Spiegazione:

```
while (corrente != null && posizioneCorrente < posizione - 1){
    corrente = corrente.getProssimo();
    posizioneCorrente++;
}
```

Condizione del ciclo:

- Il ciclo continua a muoversi lungo la lista finché entrambe le condizioni sono vere:
 - `corrente != null`: assicura che non abbiamo raggiunto la fine della lista.
 - `posizioneCorrente < posizione - 1`: verifica che non abbiamo ancora raggiunto il nodo **prima** della posizione desiderata.

Obiettivo: raggiungere il nodo corrente che si trova **immediatamente prima** della posizione in cui vogliamo inserire il nuovo elemento.

- **Istruzioni all'interno del ciclo:**
 - `corrente=corrente.getProssimo()`: sposta il riferimento corrente al nodo successivo nella lista.
 - `posizioneCorrente++`: incrementa la variabile `posizioneCorrente` per tenere traccia della posizione attuale.

Esempio di funzionamento

Si suppone di avere una lista collegata: 10 -> 20 -> 30 -> null
 Si vuole inserire un nuovo nodo 15 in **posizione 2**. Ecco come funziona il ciclo:

Stato iniziale:

- `corrente = testa (nodo 10).`
- `posizioneCorrente = 1.`

Primo ciclo:

- Verifica:
 - `corrente != null` (vero, corrente punta a 10).
 - `posizioneCorrente < posizione - 1` (`1 < 2` → falso).

Il ciclo **non si ripete**, perché `posizioneCorrente` ha raggiunto la condizione richiesta (1, il nodo immediatamente prima della posizione desiderata).

```
nuovoNodo.setProssimo(corrente.getProssimo());
```

- Questa istruzione collega il nuovo nodo (`nuovoNodo`) al nodo che si trova **immediatamente dopo** il nodo corrente (`corrente`).
- **Significato:**
 - `corrente.getProssimo()`: ottiene il riferimento al nodo successivo rispetto a `corrente`.
 - `nuovoNodo.setProssimo(...)`: imposta questo riferimento (il nodo successivo di `corrente`) come "prossimo" di `nuovoNodo`.

```
corrente.setProssimo(nuovoNodo);
```

- `corrente.setProssimo(nuovoNodo)`: fa sì che il nodo corrente punti al nuovo nodo.

```
public class Main {
    public static void main(String[] args) {
        Lista lista = new Lista();
        // Stampo la lista originale
        System.out.println("Lista originale:");
        lista.stampaLista();
        // Inserisco un elemento in posizione intermedia (tra nodo 10 e nodo 20)
        System.out.println("Inserimento in posizione 2:");
        lista.inserisciInPosizione(2, 15);
        // Stampiamo la lista dopo l'inserimento
        System.out.println("Lista dopo l'inserimento:");
        lista.stampaLista();
    }
}
```

ELIMINAZIONE IN POSIZIONE INTERMEDIA

La lista inizialmente contiene tre nodi e, utilizzando un metodo dedicato, si può eliminare un nodo in base alla posizione specificata.

```
public class Nodo {  
    private int dato;  
    private Nodo prossimo;  
    public Nodo(int dato) {  
        this.dato = dato;  
        this.prossimo = null;  
    }  
    public int getDato() {  
        return dato;  
    }  
    public Nodo getProssimo() {  
        return prossimo;  
    }  
    public void setProssimo(Nodo prossimo) {  
        this.prossimo = prossimo;  
    }  
}
```

```
public class Lista {  
    private Nodo testa; // Primo nodo della lista  
    // Costruttore: inizializza la lista con tre nodi predefiniti  
    public Lista() {  
        Nodo nodo1 = new Nodo(10);  
        Nodo nodo2 = new Nodo(20);  
        Nodo nodo3 = new Nodo(30);  
        // Collegamento dei nodi  
        nodo1.setProssimo(nodo2);  
        nodo2.setProssimo(nodo3);  
  
        // Imposta il primo nodo come testa della lista  
        testa = nodo1;  
    }  
}
```

```
// Metodo per eliminare un nodo in una posizione specifica
```

```
public void eliminaInPosizione(int posizione) {  
  
    if (testa == null) {  
        System.out.println("La lista è vuota!");  
        return;  
    }  
    if (posizione <= 0) {  
        System.out.println("Posizione non valida.");  
        return;  
    }  
    if (posizione == 1) {  
  
        // Eliminazione del nodo in testa  
        testa = testa.getProssimo();  
        return;  
    }  
}
```



```
/*
```

```
Es. testa -> 10 -> 20 -> 30 -> null
```

```
Testa punta al nodo con valore 10
```

```
testa = testa.getProssimo():
```

- testa viene aggiornato per puntare al nodo successivo (20).
- Il nodo 10 non è più collegato alla lista poiché nessun'altra variabile o nodo contiene un riferimento al nodo 10, esso è effettivamente "perso" dalla lista.
- In Java, il **garbage collector** eliminerà il nodo 10 dalla memoria poiché non ci sono più riferimenti che lo puntano.

```
*/
```

```
    Nodo corrente = testa;
```

```
    int posizioneCorrente = 1;
```

```
    // Attraversa la lista fino al nodo prima della posizione da eliminare
```

```
    while (corrente != null && posizioneCorrente < posizione - 1) {
```

```
        corrente = corrente.getProssimo();
```

```
        posizioneCorrente++;
```

```
    }
```

```
    if (corrente==null || corrente.getProssimo()==null) {
```

```
        System.out.println("Posizione supera la dimensione della lista.");
```

```
        return;
```

```
    }
```

```
    // Collegamento per saltare il nodo da eliminare
```

```
    corrente.setProssimo(corrente.getProssimo().getProssimo());
```

```
}
```

```
// Metodo per stampare la lista
```

```
public void stampaLista() {
```

```
    Nodo corrente = testa;
```

```
    while (corrente != null) {
```

```
        System.out.print(corrente.getDato() + " -> ");
```

```
        corrente = corrente.getProssimo();
```

```
    }
```

```
    System.out.println("null"); //fine della lista
```

```
}
```

```
}
```

```
corrente.setProssimo(corrente.getProssimo().getProssimo());
```

```
corrente.getProssimo():
```

- restituisce il riferimento al nodo **successivo** rispetto al nodo attualmente puntato da corrente.

```
corrente.getProssimo().getProssimo():
```

- accede al riferimento del nodo che viene dopo il successivo.
- In pratica, identifica **il nodo da saltare**, ossia il secondo nodo dopo corrente.

```
corrente.setProssimo(...):
```

- modifica il collegamento del nodo corrente (corrente) per farlo puntare **direttamente** al nodo restituito da corrente.getProssimo().getProssimo().

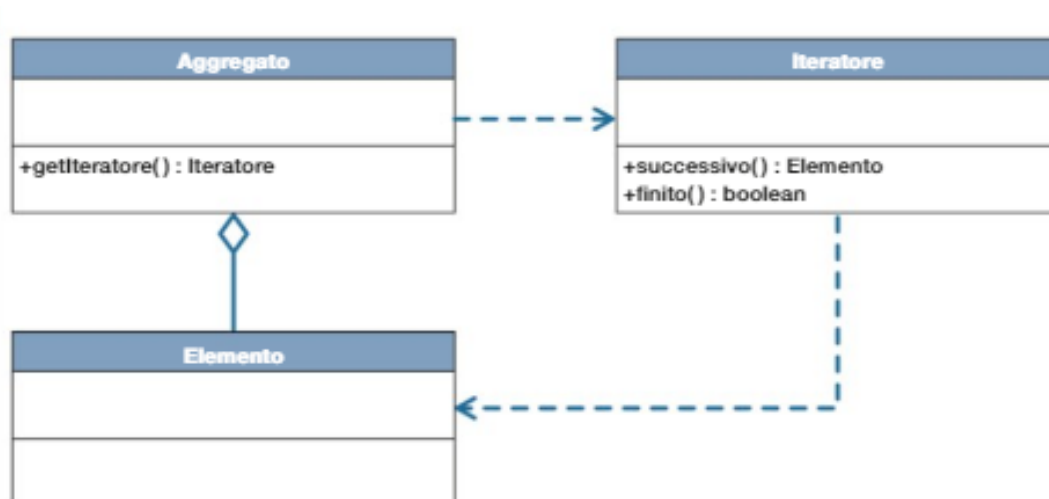
```
public class Main {
    public static void main(String[] args) {
        Lista lista = new Lista();
        System.out.println("Lista originale:");
        lista.stampaLista();
        // Elimino un elemento in posizione intermedia (es. posizione 2)
        System.out.println("Eliminazione in posizione 2:");
        lista.eliminaInPosizione(2);
        // Stampo la lista dopo l'eliminazione
        System.out.println("Lista dopo l'eliminazione: ");
        lista.stampaLista();
    }
}
```

IL PATTERN ITERATORE

Capita spesso di aver a che fare con strutture dati complesse, come ad esempio, liste, code, ecc. È buona norma accedere a queste strutture senza esporre la loro organizzazione. Inoltre, è a volte necessario percorrere la struttura in modi diversi, senza per questo caricare l'interfaccia della medesima con un numero eccessivo di metodi.

Il pattern **Iterator** ha lo scopo di fornire un oggetto (o più oggetti) per percorrere la struttura in modo sequenziale senza, però, esporre la sua struttura interna. Un iterator corrisponde al concetto di cursore (indice): il cursore punta a un elemento nella struttura e fornisce i metodi per percorrerla.

Il diagramma UML seguente fornisce una rappresentazione grafica del **Pattern Iterator**.



- **Aggregato** → contiene una collezione di elementi e offre il metodo `getIterator()`, che restituisce un oggetto di tipo **Iteratore**.
- **Iteratore** → fornisce i metodi `successivo()`, che restituisce un oggetto di tipo **Elemento**, e `finito()`, che indica se ci sono ancora elementi da iterare.
- **Elemento** → rappresenta l'unità base della collezione, ma nel diagramma non ha attributi o metodi definiti.

Relazioni tra le classi:

- **Aggregato** **compone** **Elemento**, come mostrato dal **rombo vuoto**, segnalando una relazione di aggregazione.
- **Aggregato** crea un **Iteratore**, che scorre gli **Elementi** attraverso un collegamento diretto.

- Iteratore utilizza successivo() per recuperare gli Elementi uno alla volta fino a quando finito() restituisce true, indicando la fine della sequenza.

Funzionamento: Il Pattern Iterator viene usato per separare la logica di attraversamento da quella della gestione della collezione stessa, migliorando l'**incapsulamento** e la **flessibilità** del codice.

Si supponga di avere una collezione di elementi, come una lista di nomi. Senza il Pattern Iterator, il codice che gestisce gli elementi e il codice che li attraversa sarebbero mescolati, rendendo tutto meno flessibile.

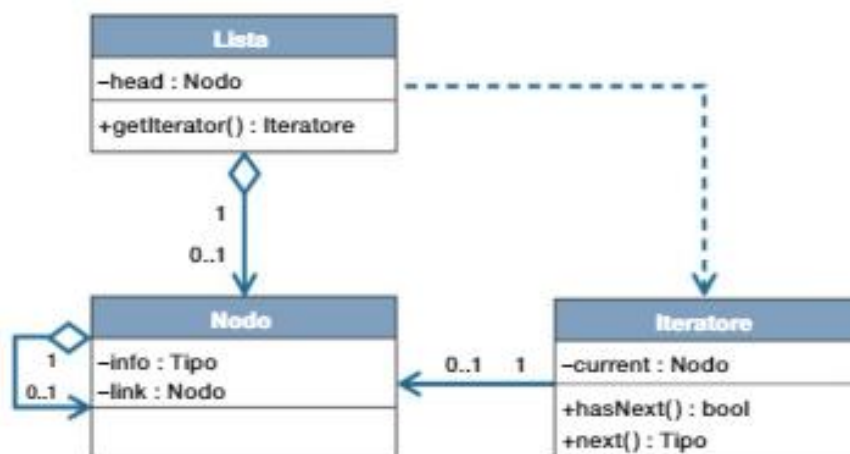
Il **Pattern Iterator** invece separa queste due responsabilità:

1. **la collezione** è responsabile solo di contenere gli elementi;
2. **l'iteratore** è responsabile di attraversare gli elementi uno alla volta.

Questa separazione migliora l'**incapsulamento**, perché la collezione non deve esporre direttamente come memorizza i dati; l'iteratore si occupa di accedere agli elementi in modo controllato. Inoltre, rende il codice **più flessibile**, perché possiamo cambiare la struttura della collezione senza dover modificare il modo in cui la attraversiamo.

Un esempio pratico: si pensi a una playlist musicale. La playlist contiene le canzoni, ma non si preoccupa di come le scorrere. L'iteratore è come il "tasto avanti" del lettore musicale: richiama la prossima canzone, senza sapere come la playlist le tiene in memoria. Questo permette di avere playlist diverse, ma sempre lo stesso modo di passare da una traccia all'altra.

Un esempio più specifico è rappresentato mediante il diagramma UML riportato di seguito.



La classe **Iteratore** permette di scorrere la lista collegata **senza esporre direttamente** la struttura interna dei nodi. Questa classe ha:

- **un riferimento** (current): mantiene traccia del nodo attuale mentre attraversiamo la lista;
- **un metodo** hasNext(): verifica se ci sono altri elementi da leggere;
- **un metodo** next(): restituisce il valore del nodo corrente e passa al successivo.

La **Lista** è la classe che gestisce l'intera sequenza di **Nodo**. In una lista collegata:

- il primo elemento (head) è il punto di partenza della lista;
- può contenere un metodo per aggiungere/eliminare elementi;
- fornisce un iteratore per attraversare gli elementi.

La classe **Nodo** **contiene un valore** (info) e **un riferimento al nodo successivo** (link). Questo riferimento stabilisce una **relazione riflessiva**, dove ogni nodo punta a un altro nodo della stessa classe.

Se abbiamo i nodi $A \rightarrow B \rightarrow C$, allora il funzionamento è il seguente:

1. A.link punta a B (quindi A è collegato a B).
2. B.link punta a C (quindi B è collegato a C).
3. C.link punta a null (ultimo nodo della lista, non ha successivi).

Si veda come esempio il programma *AgendaContatti* (caricato in Teams) che mostra come implementare il Pattern Iterator.